

Robert 莊

Mobile App Solution

專案作品集 Since 2024-11

16

年開發經驗

9

技術專案

3

核心平台



完整互動式作品集 (含影片演示)

<https://pub-7ffd7755217c4c84be4b81e3cddd2ad0.r2.dev/index.html>

專案目錄

1.

Server級 AI 移植專案 | Flutter SDK化

★★★

On-device AI、Flutter Plugin 化、iOS/Android 跨平台 (C++/FFI)

2.

企業級員工考勤管理App

★★

MVVM 架構實踐、離線資料同步、人臉識別整合、Beacon 室內定位

3.

企業級設施管理App



模組化架構、Firebase 推播整合、多語系支援、即時資料同步

4.

企業級實驗室管理App



Flutter 跨平台開發、BLoC 架構設計、多層權限控制、即時通知推播

5.

物業管理用戶端iOS版App



維護公司歷史悠久大型iOS專案

6.

雪霸國家公園 iOS App



Objective-C 原生開發、已讀未讀推播系統、位置服務整合、RESTful API 串接

7.

國家公園 iOS App



iOS 原生開發、RESTful API 整合、推播通知系統、位置服務、UI/UX 重構

8.

跨域緊急救援-PHP推播系統



PHP Laravel 快速上手、Legacy 系統分析、多平台推播修復、緊急專案交付

9.

AI智能保全 App



Flutter 架構重構、AI 模型管理、FFI 整合、協力開發交付

1. Server級 AI 移植專案 | Flutter SDK化

On-device AI、Flutter Plugin 化、iOS/Android 跨平台 (C++/FFI)

運用技術：On-device AI / Flutter / iOS / Android / C++ / FFI / ONNX Runtime / OpenCV

將原本在 Server AI辨識模組移植至flutter行動端，單張人臉運算約500毫秒體驗即時、不卡頓，並且封裝成 SDK，方便套用到任意flutter專案且無網也能辨識。

背景與目標

- 主管希望能在無網的環境下也能使用臉部辨識功能，於是才有了這個將Server辨識AI移植到行動裝置的專案
- 接手的時候原本的flutter工程師已經離職，從他的專案可以看到他試圖在flutter層引用opencv 以及 onnxruntime框架想要直接在 flutter 層直接運算AI辨識
- 在 flutter 層直接運算AI辨識運算的結果相當糟糕，且在無法使用 flutter 異步多工的狀況下，執行結果就是UI直接鎖死3~5秒鐘

我的角色

- 團隊角色：影像辨識團隊由我與另一位Server(python)工程師共同負責
- 由我主導：透過 AI 多次的資料搜查與確認，且與同事討論之後確定要使用 FFI 架構，由我同事完成 python 轉 C++的部分，剩下部分由我完成後續C++與flutter的介接

關鍵挑戰

- **第一階段**：成功使用FFI介面呼叫了C++程式
- **第二階段**：重新將專案建構成 flutter plugin 架構，且為了能讓外部可使用將C++程式碼封裝成動態庫
- **第三階段**：app應用端引用了 plugin 架構後遇到apple的巢狀式動態庫(Nested Dynamic Frameworks)問題，只好再將C++編譯成靜態庫，才能通過apple的程式碼檢驗
- **總結**：C++ 不是我的專長，尤其靜態庫的編譯/鏈結難度更是超出我的想像；但最後還是能打通最後關卡完成交付的任務

效能與成果

- 直接透過FFI介面在C++運算回來的結果從5秒 > 約500毫秒
- 解決flutter層級無法使用多工造成的畫面凍結的問題
- 最終完成了 flutter plugin 架構，任意專案都可以方便引用，重複利用性高
- C++ 封裝成靜態庫，提供公司外部使用也不擔心C++程式內容被看到
- On-device AI 趨勢：無網路也能運算，且減少雲端運算 loading，歐美對於AI的法律規範已逐步成型

2. 企業級員工考勤管理App

MVVM 架構實踐、離線資料同步、人臉識別整合、Beacon 室內定位

運用技術：Flutter / Provider / Hive / Firebase / Beacon / Dio / FFI

開發一套完整的員工考勤管理系統，實現離線優先架構，確保無網路環境下仍可正常打卡並自動同步。整合人臉識別與 Beacon 技術達成精準考勤，採用MVVM 架構確保程式碼可維護性，並實現 401 錯誤自動處理與 Token 刷新機制。

背景與角色

擔任企業人臉辨識考勤系統 Flutter App 獨立開發者，負責整合考勤與人臉辨識雙後端架構。開發核心功能包含：ONNX 模型人臉辨識、Beacon 室內定位打卡、離線優先作業機制，以及時間竄改檢測、離線數據完整性驗證等多層防偽技術。

關鍵挑戰

離線獨立作業

設計了網路感知式離線打卡系統，實現基於 Connectivity 監聽的智能模式切換機制，當檢測到離線狀態時自動啟動獨立的本地打卡流程，搭配離線數據持久化與增量同步策略，建立離線優先架構確保核心打卡功能在任何網路條件下都能穩定運作，有效解決弱網環境下的數據一致性問題。

企業級混合推播系統重構

針對公司既有 APNS 原生架構設計 Method Channel 雙向通信機制整合 Firebase FCM，克服 Flutter Firebase Plugin 與企業傳統推播服務的相容性限制，透過平台特定實作統一 iOS/Android 推播機制差異，實現原生推播與本地通知的無縫整合，確保跨平台通知顯示的一致性與穩定性。

成果展示



主要功能演示影片：

<https://raw.githubusercontent.com/robert008/resume-assets/main/attendance-app/demo.MOV>

 離線打卡演示影片：

https://raw.githubusercontent.com/robert008/resume-assets/main/attendance-app/offline_punch_demo.MOV

 ABM企業部署截圖：

https://raw.githubusercontent.com/robert008/resume-assets/main/attendance-app/app_store.png

3. 企業級設施管理App

模組化架構、Firebase 推播整合、多語系支援、即時資料同步

運用技術：Flutter / BLoC Pattern / Firebase Messaging / WebSocket / i18n

將會議室、維修、餐飲等功能模組化開發，每個功能獨立 BLoC 管理狀態。整合 Firebase 推播通知與 WebSocket 即時通訊，支援繁簡中英三語切換，透過 WebSocket 實現即時通知。

角色與背景

7-8人團隊協作開發，我與另一位 Flutter 工程師負責 App 端開發。作為後期加入成員，需從原本熟悉的 Provider 狀態管理模式，快速轉換適應團隊既有的 BLoC + Riverpod 混合架構及相關技術棧。

關鍵挑戰

大型專案架構管理

約 12 萬行程式碼、531 個檔案的複雜專案開發，採用 BLoC + Riverpod 混合狀態管理架構處理複雜業務邏輯，實現會議室、設備維護、實驗室、餐飲、包裹等多功能模組化整合設計，確保大型專案的可維護性與擴展性。

國際化多語言支援

使用 flutter_i18n 實現英文、繁體中文、簡體中文的完整多語言架構與動態語言切換功能，妥善處理不同語言的 UI 適配和文字長度變化問題，確保跨語言使用體驗的一致性。

企業級複雜功能實現

建立多廠區支援體系涵蓋不同工廠、樓層、部門的權限控制，設計精細權限管理機制處理相機、儲存、通知等多重權限控制，整合設備保養、會議預約、實驗室借用等複雜企業流程，提供完整的企業數位化解決方案。

成果展示

 功能演示影片：

<https://raw.githubusercontent.com/robert008/resume-assets/main/facility-mgmt/demo.m4v>

 專案架構圖：

https://raw.githubusercontent.com/robert008/resume-assets/main/facility-mgmt/project_structure.png

 TestFlight 部署記錄：

https://raw.githubusercontent.com/robert008/resume-assets/main/facility-mgmt/apple_store.png

4. 企業級實驗室管理App

Flutter 跨平台開發、BLoC 架構設計、多層權限控制、即時通知推播

運用技術：Flutter / BLoC / RESTful API / Firebase Messaging / QR Code

企業級設施管理的衍伸案，開發實驗室設備與空間管理系統，支援設備借用、維護保養、故障修繕等完整流程。實作多層級權限控制，整合 Firebase 推播即時通知，支援四語系切換（繁中/簡中/英文/泰文）。

角色與背景

5人敏捷開發團隊成員，與 3 名 Web 前端和 1 名 Flutter 開發夥伴協作，原企業級設施管理 App 因外包商與業主溝通分歧，急需在不到一個月的極限時程內將實驗室管理模組獨立成單一應用。

關鍵挑戰

面臨極短時程內完成專案分離、重構與測試的時程壓力，在關鍵開發人員流失導致人力短缺的困境下維持開發進度，需將緊密耦合的模組安全分離避免功能缺失，同時在時間壓縮下確保新應用的穩定性與可靠性，並在複雜的外包關係中保持有效的技術溝通協調。

成果展示

 功能演示影片：

<https://raw.githubusercontent.com/robert008/resume-assets/main/facility-lab-mgmt/demo.mov>

 TestFlight 交付歷程：

https://raw.githubusercontent.com/robert008/resume-assets/main/facility-lab-mgmt/testflight_record.png

5. 物業管理用戶端iOS版App

維護公司歷史悠久大型iOS專案

運用技術：Objective-C / Swift / Flutter

維護公司歷史悠久大型iOS專案，錯誤修護以及加入一些新功能

角色與背景

獨立維護公司運作多年的 iOS App，該專案採用 Objective-C、Swift、Flutter 混合技術架構，經過多年發展和多位開發者不同風格的程式碼累積，由於開發時程緊迫缺乏完整文件記錄與規格說明，需在複雜的混合代碼環境中進行維護和功能開發。

關鍵挑戰

技術債務累積

專案經過多人接手開發，累積了不同編碼風格的混合程式碼，同時使用 Objective-C、Swift、Flutter 三種技術架構，導致部分程式碼（如複雜的 delegate 鏈）已無人完全理解，形成嚴重的技術債務和維護困難。

維護模式的限制

在時間壓力下進行 bug 修復，只能採取最小幅度修改策略，避免大規模架構重構以防影響其他功能模組，形成保守維護模式的技術約束。

多Target複雜度管理

維護包含 10+ 個 target 的複雜 iOS App，需針對每個客戶的差異化需求進行客製化開發，任何程式碼改動都必須確保不影響其他品牌版本的功能穩定性，增加了開發和測試的複雜度。

成果展示



App Store 展示：

<https://raw.githubusercontent.com/robert008/resume-assets/main/crm-ios/appstore.png>

6. 雪霸國家公園 iOS App

Objective-C 原生開發、已讀未讀推播系統、位置服務整合、RESTful API 串接

運用技術：Objective-C / UIKit / Core Location / Push Notifications

台灣雪霸國家公園官方入園管理應用，提供登山客空中報到、推播訊息、地圖導航、安全回報等功能。實作智能推播已讀未讀狀態管理，整合 Core Location 定位服務，支援離線地標資料查詢與 AR 擴增實境地標可視化。

角色與背景

獨立維護既有 iOS 原生應用 App。需在原有應用架構存在技術債務問題上協助客戶解決問題。

關鍵挑戰

面對高度耦合的 Objective-C 舊代碼架構進行功能增強，需在不破壞現有穩定功能的前提下實現複雜的推播狀態管理邏輯。

成果展示

 功能演示影片：

<https://raw.githubusercontent.com/robert008/resume-assets/main/spnational-park-ios/demo.mov>

7. 國家公園 iOS App

iOS 原生開發、RESTful API 整合、推播通知系統、位置服務、UI/UX 重構

運用技術：Objective-C / UIKit / Core Location / Push Notifications

原雪霸國家公園官方行動應用程式改寫成新 App，提供登山者完整的入園服務體驗，包含空中報到簽退、步道地圖導航、緊急求救簡訊、推播訊息通知等核心功能。

角色與背景

需獨立開發新 App，將原本的雪霸 App 專案雪霸國家公園APP的系統重構與平台化改造，將原有單一公園應用擴展為支援三個國家公園（雪霸、太魯閣、玉山）的統一平台，包含API架構重構、多租戶支援、UI模組化等工作。

關鍵挑戰

將API從 SOAP 協定的 .asmx 服務遷移至 RESTful API，同時需要重新設計推播通知的讀取狀態管理機制。面對缺乏完整技術文件的遺留程式碼，必須透過逆向工程理解原有邏輯，並在維持向後相容性的前提下實現功能優化。

成果展示



平台化重構演示：

<https://raw.githubusercontent.com/robert008/resume-assets/main/national-park-ios/demo.mov>

8. 跨域緊急救援-PHP推播系統

PHP Laravel 快速上手、Legacy 系統分析、多平台推播修復、緊急專案交付

運用技術：Laravel 9.x / Firebase FCM / Apple APN / PHP / Legacy Code Analysis

在原開發者離職且無文件的緊急狀況下，承接企業級推播系統維護任務。以 App 開發背景快速學習 PHP Laravel 框架，在既有 App 開發期程內同步完成推播系統修復。

背景與角色

面臨團隊技術斷層的緊急狀況：公司多 App 共用推播系統因原開發者離職且未留下開發文件，導致後端工程師無法有效維護商業邏輯與推播流程。在主管要求限時交付的壓力下，跳出 Flutter App 開發舒適圈，承接從未接觸過的 PHP Laravel 後端推播開發任務，並需在原有 App 開發期程內同步完成兩項專案交付。

關鍵挑戰

零基礎 PHP 快速戰力化

面對完全陌生的 PHP Laravel 技術棧與無文件 Legacy 系統，透過逆向工程分析既有程式碼架構，快速理解多租戶推播服務的核心邏輯與資料流程。在緊迫時程壓力下找出關鍵的程式區段，且要在不影響原先程式運作的前提下，加入新的 android 推播程式(因為原先的 Android Legacy FCM Key已經不能再使用)，最終成功打通 Android/iOS 雙平台推播鏈路，確保專案如期交付。

成果展示

 推播系統架構圖：

https://raw.githubusercontent.com/robert008/resume-assets/main/emergency-push/project_structure.png

9. AI智能保全 App

Flutter 架構重構、AI 模型管理、FFI 整合、協力開發交付

運用技術：Flutter / BLoC / Clean Architecture / FFI / AI Model Management

在不影響主系統穩定性前提下，完整整合企業級臉部辨識系統。從既有專案移植完整功能模組，實作 AI模型動態下載、權限分層控制，成功加入臉部驗證機制。

背景與角色

協力開發任務，需將公司考勤專案的臉部辨識系統完整移植到智能安全守護平台。

關鍵挑戰

不同架構模組化整合

面對既有專案的 MVVM 架構與目標專案的 BLoC Clean Architecture 差異，需要在不破壞原有設計模式的前提下完成整合。透過分析兩個架構的數據流差異，設計適配層將臉部辨識模組無縫整合到認證 BLoC 中，並實作 AI 模型狀態管理、權限分層檢查、登出安全清理等機制。成功建立符合 Clean Architecture原則的企業級臉部辨識系統，確保代碼可維護性與系統穩定性。

成果展示



AI整合演示影片：

<https://raw.githubusercontent.com/robert008/resume-assets/main/security-app/demo.MOV>

